Réseaux : cloud et Internet des objets

Séance 2 : conteneurs et Docker Compose

Chérifa Boucetta

Cherifa.boucetta@univ-eiffel.fr

Master 2 RSA · 2026–2027



QCM de rappel : séance 1

Wooclap

Rejoindre la session

wooclap.com

Code de l'événement: **QODFKIL**

6 questions, sans note

- Cloud, virtualisation et réseau virtuel
- Les notions de la séance 1 servent de base aux conteneurs

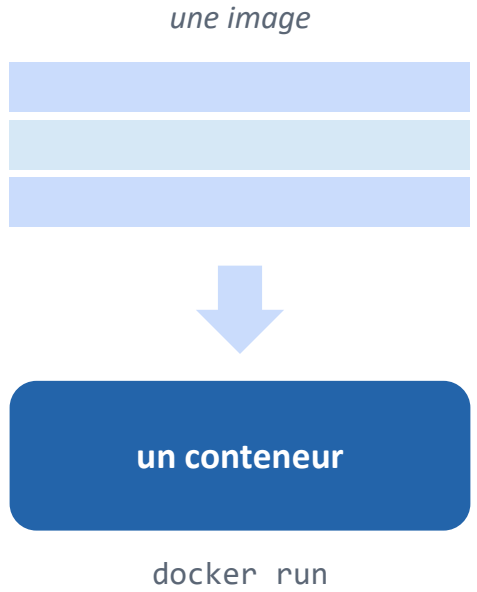
https://app.wooclap.com/QODFKIL?from=instruction-slide

Plan de la séance

1. **Docker : images et conteneurs**
   Moteur, images en couches, cycle de vie, réseau

2. **Construire ses images**
   Dockerfile, construction, bonnes pratiques

3. **Plusieurs services avec Compose**
   YAML, services, réseaux, volumes, démarrage ordonné



Pourquoi Docker ?

Le problème

- Déployer une application à la main est pénible et peu reproductible
- Les dépendances de deux projets peuvent se perturber

La réponse

- Packager l'application et ses dépendances dans un processus isolé : un conteneur
- Le même résultat sur toute machine qui exécute Docker

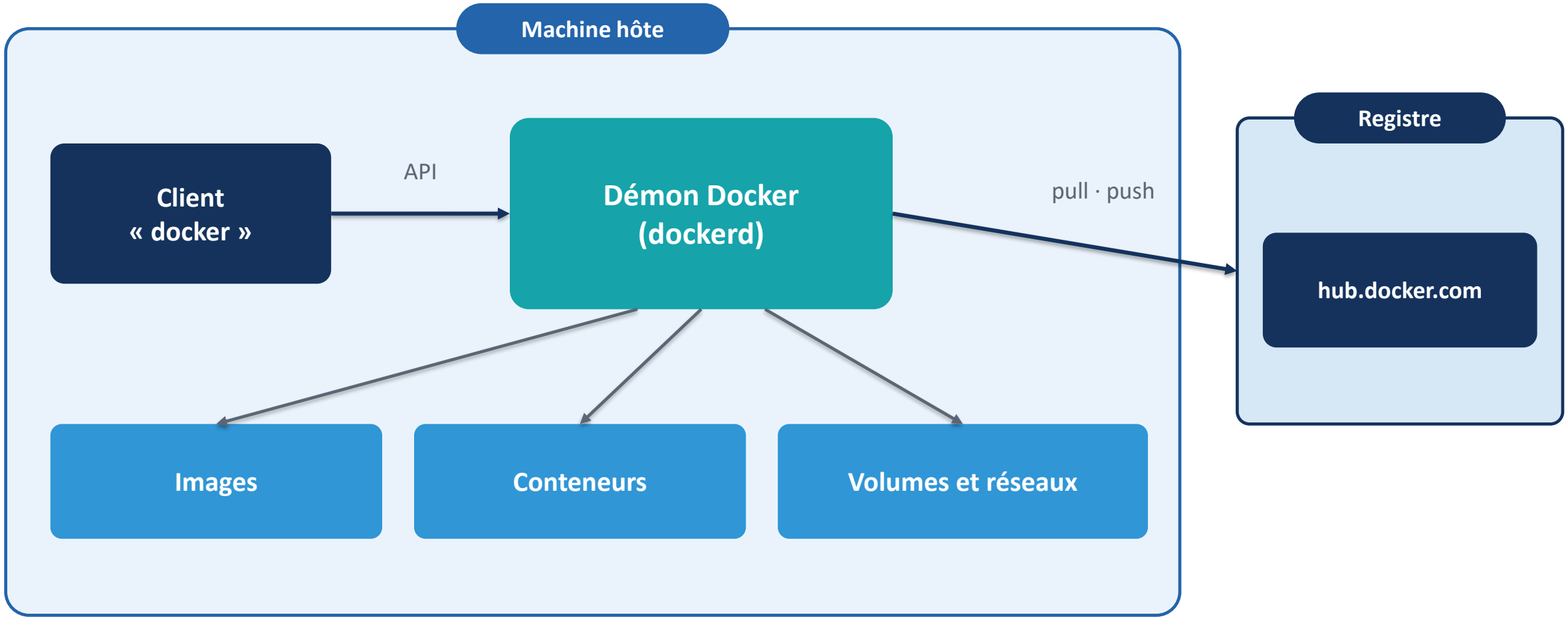
Une image

Un modèle pour créer un conteneur, comme un programme

Un conteneur

L'instance qui tourne sur la machine, comme un processus

Docker Engine



Documentation : docs.docker.com

Image, conteneur, registre, volume

Image

- Modèle en lecture seule, composé de couches
- Code, outils d'exécution, dépendances, bibliothèques

Conteneur

- Instance en cours d'exécution d'une image
- Ajoute une couche en lecture/écriture, éphémère

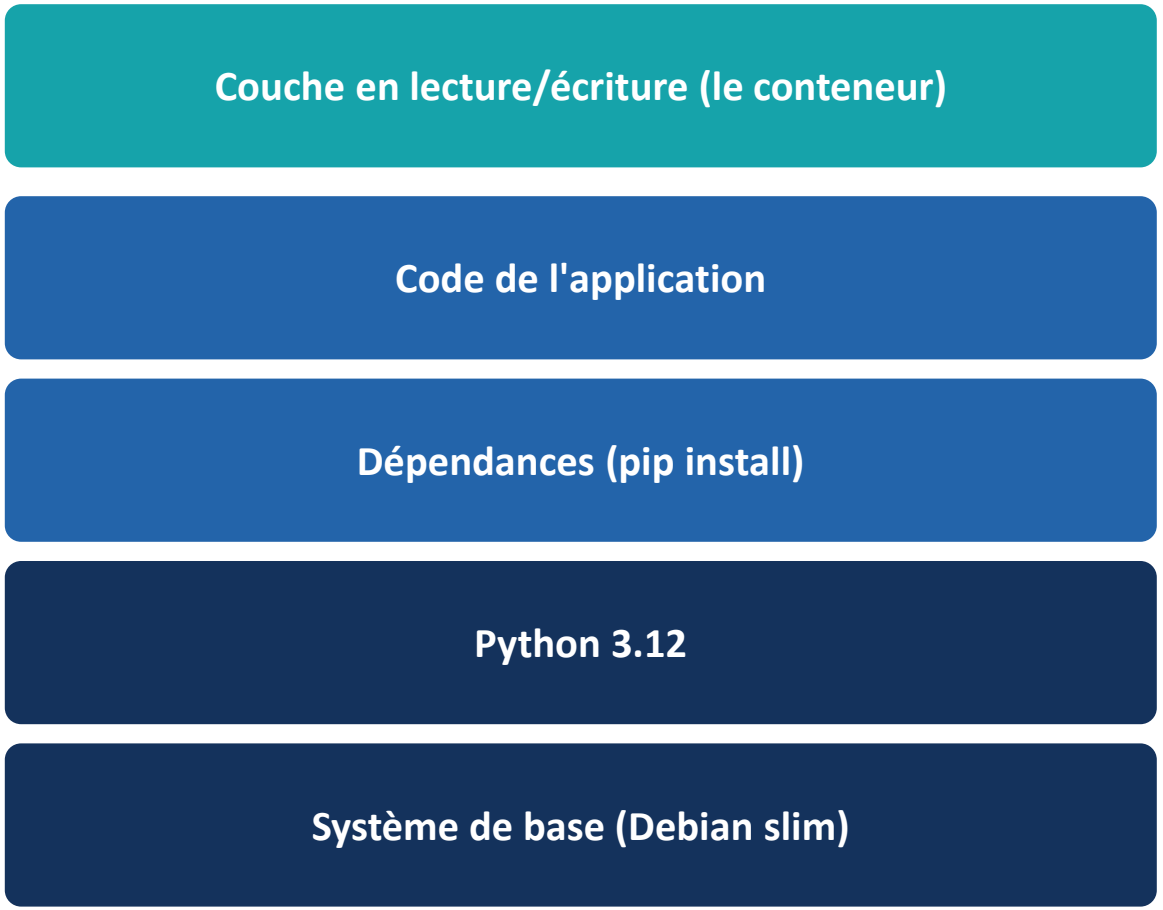
Registre

- Dépôt d'images à télécharger
- Dépôt officiel : hub.docker.com

Volume

- Dossier partagé entre l'hôte et le conteneur
- Assure la persistance des données

Les couches d'une image



Couches de l'image : en lecture seule

- Chaque instruction d'un Dockerfile crée une couche
- Les couches sont mises en cache et partagées entre images
- Le conteneur écrit dans sa propre couche : elle disparaît avec lui
- Ce qui doit survivre va dans un volume

Installer Docker

Installation à partir du dépôt officiel, puis vérification

```
$ sudo apt-get update
$ sudo apt-get install docker-ce docker-ce-cli containerd.io docker-compose-plugin
$ sudo service docker start
$ sudo docker run hello-world
```

Variante rapide

sudo apt install docker.io docker-compose-v2, puis sudo usermod -aG docker $USER pour éviter sudo

La commande docker

L'aide affiche quatre zones

- **Usage :** comment utiliser la commande
- **Options :** conditions spécifiques d'usage
- **Management Commands :** éléments sur lesquels agir
- **Commands :** actions à réaliser

Obtenir de l'aide : --help

```
$ docker --help
$ docker image --help
$ docker run --help
```

- L'aide existe à chaque niveau
- Même principe pour docker compose --help

Les commandes essentielles

Action	Commande
Lancer un conteneur	`docker run -ti --name myalpine alpine`
Lister les conteneurs actifs	`docker ps`
Lister tous les conteneurs	`docker ps -a`
Arrêter un conteneur	`docker stop myalpine`
Redémarrer un conteneur	`docker start myalpine`
Supprimer un conteneur	`docker rm myalpine`

Inspecter et manipuler

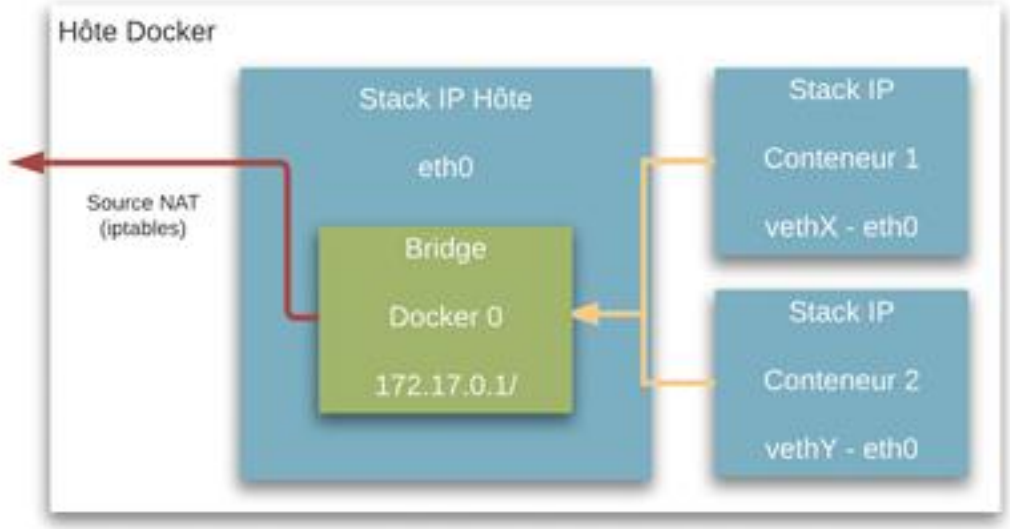
Action	Commande
Ouvrir un terminal dans un conteneur	`docker exec -ti myalpine sh`
Afficher les propriétés (JSON)	`docker inspect myalpine`
Afficher les journaux	`docker logs myalpine`
Lister les images	`docker image ls`
Télécharger une image	`docker pull alpine`
Copier un fichier vers un conteneur	`docker cp fichier.txt myalpine:/tmp`

Quelques paramètres de docker run

Paramètre	Action
`-d`	exécuter en arrière-plan
`-ti`	ouvrir un terminal interactif
`--name nom`	nommer le conteneur
`--rm`	supprimer le conteneur à sa sortie
`-p hôte:conteneur`	publier un port
`-v hôte:conteneur`	monter un dossier ou un volume
`-e VARIABLE=valeur`	définir une variable d'environnement
`--network nom`	choisir le réseau

Le réseau Docker

- Par défaut, un conteneur est relié au bridge docker0
- Sous-réseau habituel : 172.17.0.0/16, passerelle 172.17.0.1
- C'est le br0 du TP 1 : veth, bridge, NAT
- Réseau bridge par défaut : accès par adresse IP
- Réseau défini par l'utilisateur : accès par le nom du conteneur



Docker crée aussi une règle de NAT pour la sortie vers Internet



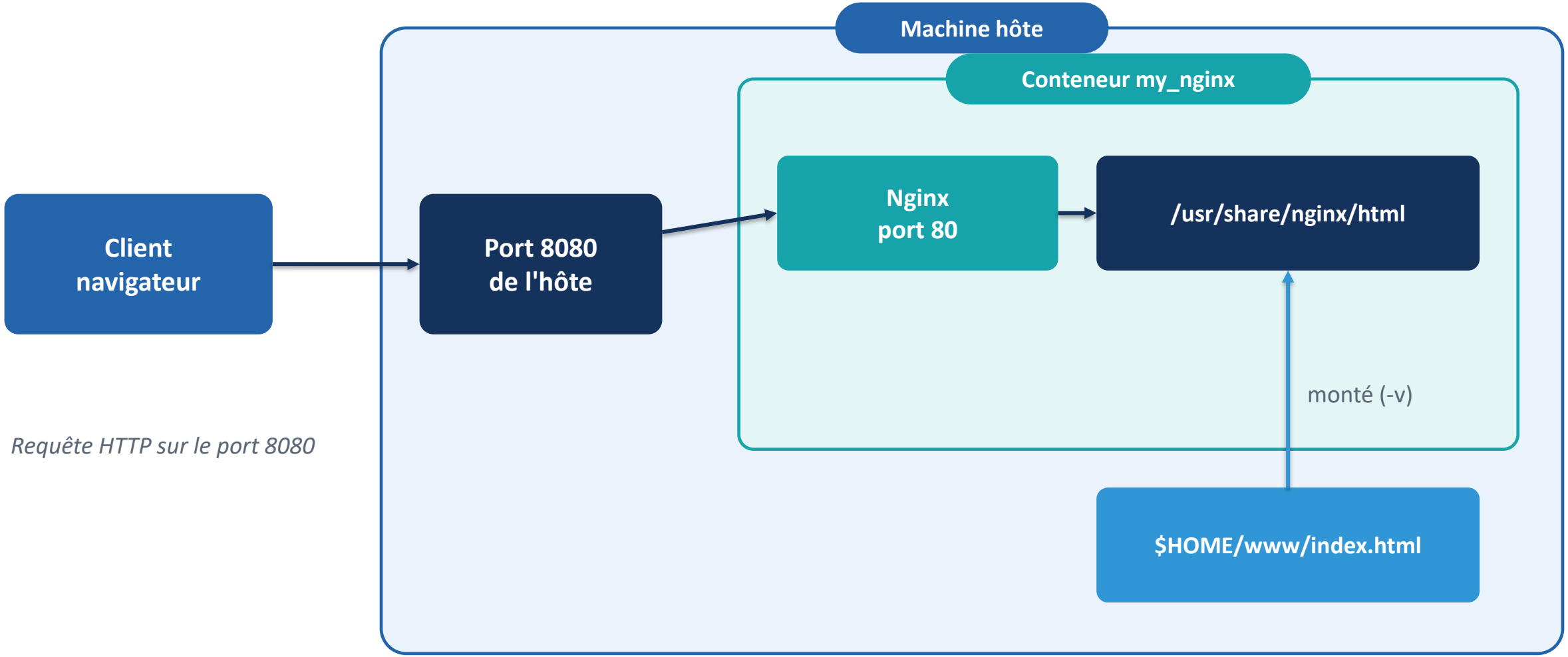
Exemple : un serveur Nginx

Un site web servi depuis un dossier de l'hôte

```
$ mkdir ~/www
$ docker run -d --rm -p 8080:80 -v $HOME/www:/usr/share/nginx/html --name my_nginx nginx:latest
```

- **-p 8080:80 :** le port 80 du conteneur devient le port 8080 de l'hôte
- **-v $HOME/www:/usr/share/nginx/html :** le dossier de l'hôte est monté dans le conteneur
- **--rm :** le conteneur est supprimé à son arrêt
- **--name my_nginx :** nom du conteneur ; nginx:latest : image la plus récente

Exemple : le trajet d'une requête



Requête HTTP sur le port 8080

TD : exercices 1 et 2

1

Cycle de vie et volumes

Créer, arrêter, supprimer un conteneur ; persister des données

`docker run -ti --name c1 alpine` · `docker ps -a` · `docker volume create mydata`

Un conteneur est éphémère : ce qui doit survivre va dans un volume.

2

Nginx, port et volume

Publier un site depuis un dossier de l'hôte

```
docker run -d --name monsitedocker -p 8080:80 -v ~/www:/usr/share/nginx/html nginx
```

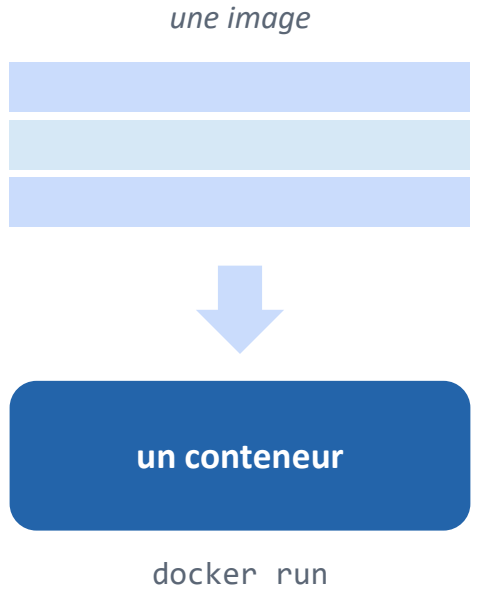
-p ouvre un port, -v partage un dossier : le site vit chez l'hôte.

Plan de la séance

1. **Docker : images et conteneurs**
   Moteur, images en couches, cycle de vie, réseau

2. **Construire ses images**
   Dockerfile, construction, bonnes pratiques

3. **Plusieurs services avec Compose**
   YAML, services, réseaux, volumes, démarrage ordonné



Pourquoi un Dockerfile ?

- Déployer une application Flask à la main à chaque fois est pénible
- Avec Docker, chaque projet est déjà isolé dans son conteneur
- Le Dockerfile est un fichier texte qui décrit toutes les commandes pour assembler l'image
- Il est construit avec docker build
- L'image obtenue est partageable et fonctionne sur toutes les plateformes supportées

Objectif

Empaqueter l'application et ses dépendances dans une image reproductible

Les instructions du Dockerfile

Instruction	Rôle
`FROM`	définit l'image de base
`WORKDIR`	définit le répertoire de travail
`COPY / ADD`	copie des fichiers ou des répertoires dans l'image
`RUN`	exécute une commande pendant la construction
`ENV / ARG`	variable d'environnement / variable de construction
`EXPOSE`	indique le port d'écoute du conteneur
`USER`	définit l'utilisateur d'exécution
`VOLUME`	crée un point de montage
`CMD / ENTRYPOINT`	commande exécutée au démarrage du conteneur
`LABEL`	ajoute des métadonnées à l'image

Exemple : une application Flask

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 5000
CMD ["python", "app.py"]
```

- Image de base légère et version précise
- Les dépendances d'abord : la couche est réutilisée tant qu'elles ne changent pas
- Le code ensuite, car il change souvent

Du Dockerfile à l'image

```
$ docker build -t api-vols .
$ docker image history api-vols
$ docker run -d -p 5000:5000 api-vols
```

- -t donne un nom à l'image
- Le point final est le contexte de construction : le dossier courant
- Chaque instruction produit une couche, mise en cache

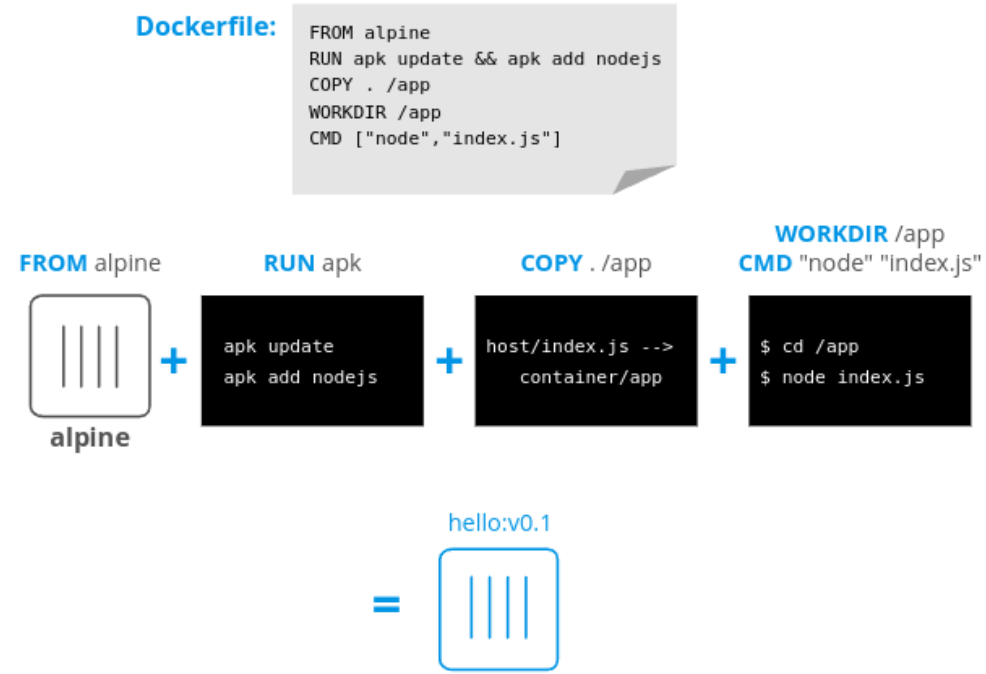


Illustration de l'ancien cours

Bonnes pratiques

1 Image de base légère

slim ou alpine, avec une version précise plutôt que latest

2 Ordre des instructions

dépendances d'abord, code ensuite, pour le cache

3 Fichier .dockerignore

exclure .git, .env et les fichiers inutiles

4 Utilisateur non-root

USER pour ne pas exécuter l'application en administrateur

5 Un processus par conteneur

un service par conteneur, combinés avec Compose

6 Aucun secret dans l'image

ni dans le Dockerfile ni dans le dépôt

TD : exercice 3

3

Dockerfile

Empaqueter une API Flask dans une image

`docker build -t api-vols .` · `docker run -d --name api -p 5000:5000 api-vols`

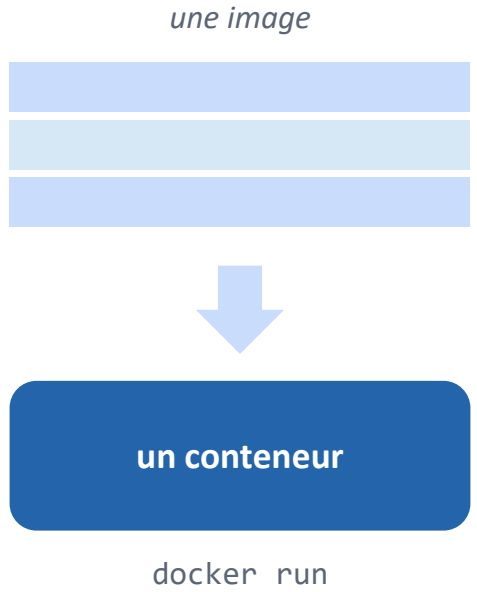
Le Dockerfile rend la fabrication reproductible ; l'ordre des instructions commande le cache.

Plan de la séance

1. **Docker : images et conteneurs**
   Moteur, images en couches, cycle de vie, réseau

2. **Construire ses images**
   Dockerfile, construction, bonnes pratiques

3. **Plusieurs services avec Compose**
   YAML, services, réseaux, volumes, démarrage ordonné



Pourquoi Docker Compose ?

- Lancer plusieurs conteneurs liés, avec réseaux et volumes, demande des commandes lourdes
- Cela devient ingérable avec beaucoup de services
- Compose décrit toute l'application dans un fichier YAML
- C'est de l'Infrastructure as Code : le fichier se versionne et se rejoue

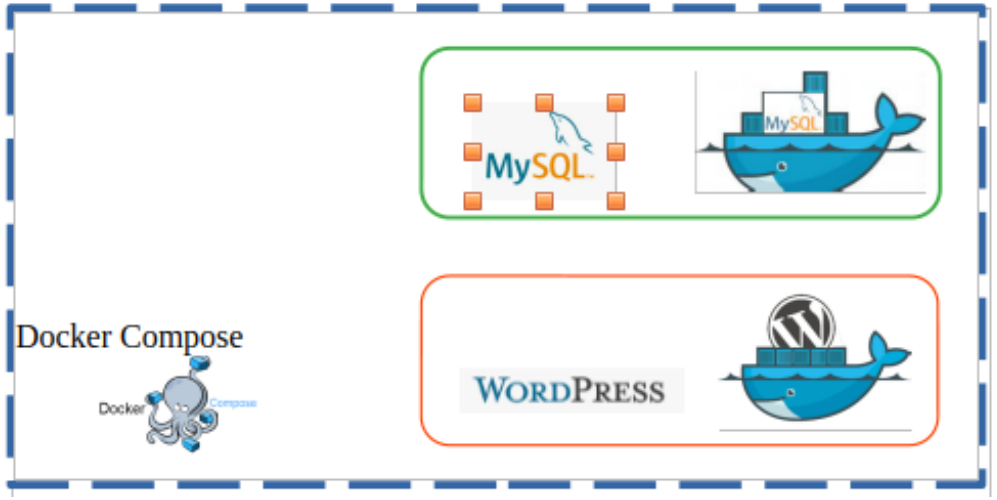


Illustration de l'ancien cours : WordPress et MySQL

Le format YAML

- Une représentation de données, fichiers .yml ou .yaml
- L'indentation structure le fichier : deux espaces, jamais de tabulation
- Les listes se présentent avec des tirets
- Paires clé: valeur, séparées par deux-points
- Un commentaire commence par #
- Proche de JSON, sans accolades

```
services:
  db:
    image: mysql:8.0
    volumes:
      - db_data:/var/lib/mysql
# un commentaire
```

Structure d'un fichier Compose

services

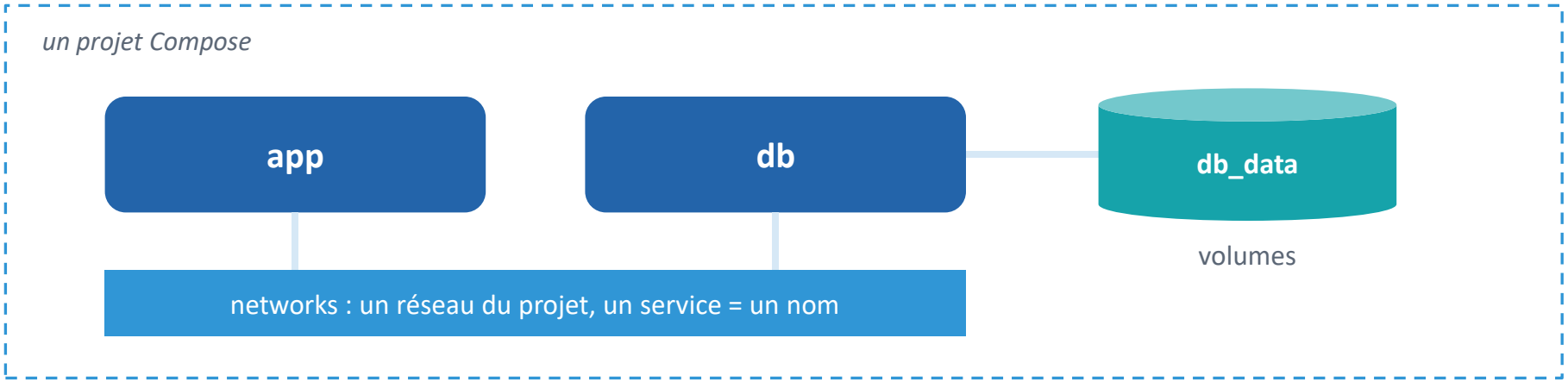
Les conteneurs à mettre en service, avec leurs options

networks

Les réseaux à créer ou à utiliser

volumes

Les volumes à créer ou à utiliser

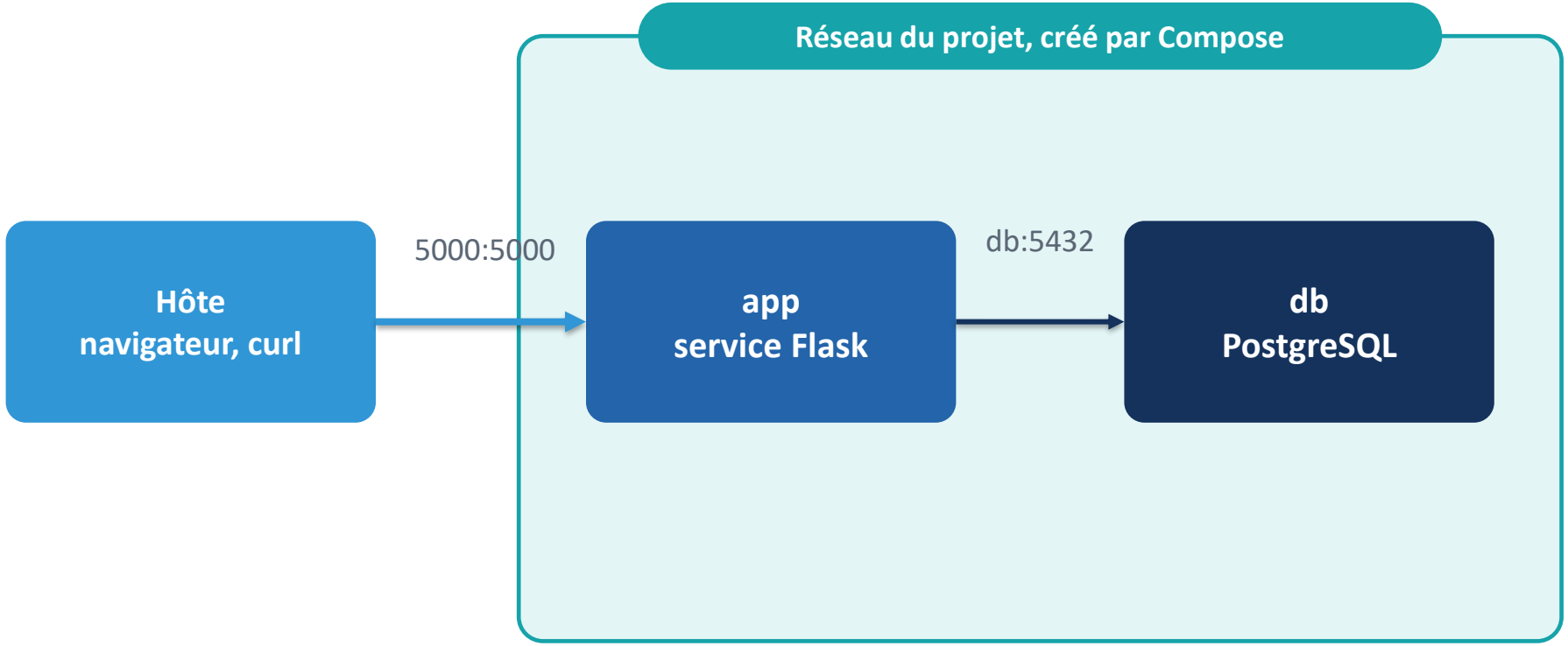


Exemple : WordPress et MySQL

```
services:
  db:
    image: mysql:8.0
    environment:
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wp
      MYSQL_PASSWORD: ${WP_PASSWORD}
      MYSQL_RANDOM_ROOT_PASSWORD: "1"
    volumes: [db_data:/var/lib/mysql]
  wordpress:
    image: wordpress
    depends_on: [db]
    ports: ["8000:80"]
    environment:
      WORDPRESS_DB_HOST: db:3306
      WORDPRESS_DB_USER: wp
      WORDPRESS_DB_PASSWORD: ${WP_PASSWORD}
      WORDPRESS_DB_NAME: wordpress
volumes:
  db_data:
```

- Deux services, dans le réseau créé par défaut par Compose
- Un volume conserve la base de données
- Un seul port publié : celui de WordPress
- Le mot de passe vient du fichier .env

Les réseaux dans Compose



- Chaque service est joignable par son nom : db, app
- Seuls les ports publiés (ports:) sont accessibles depuis l'hôte : db n'en publie aucun

Le workflow Compose

Commande	Effet
`docker compose up -d`	démarrer en arrière-plan
`docker compose up --build`	reconstruire les images, puis démarrer
`docker compose ps`	état des services
`docker compose logs -f`	journaux, en continu
`docker compose exec service cmd`	exécuter une commande dans un service
`docker compose stop / start`	arrêter / redémarrer sans supprimer
`docker compose down`	supprimer conteneurs et réseaux
`docker compose down -v`	supprimer aussi les volumes

TD : exercices 4 et 5

4

Compose : Flask et PostgreSQL

Deux services, un réseau, un volume, un healthcheck

`docker compose up --build -d` · `docker compose ps` · `docker compose logs app`

Compose crée un réseau par projet : chaque service est joignable par son nom.

À retenir

Docker

Une image est un modèle, un conteneur en est l'instance. Les données à conserver vont dans un volume.

Dockerfile

Il décrit comment fabriquer une image. L'ordre des instructions détermine le cache.

Compose

Un fichier décrit services, réseaux et volumes. Les services se joignent par leur nom.